

# プログラミング言語論

## 第10回 オブジェクト指向

篠埜 功

# プログラムの分割

大きなプログラムは、分割して開発するのがよい。  
手続きは分割の最も基本的なものである。

Module(モジュール)は、関連する変数、手続き、  
型を一つにまとめるものである。Modula-2 (Pascal  
の後継)ではモジュールが使える。

手続きが処理するデータの型を一つのモジュール  
内に入れ、データ型の実装を隠す。

# モジュール

モジュールは宣言(型、変数、手続き等)をグループ化。  
モジュールから型をexportすることによりユーザ定義の  
データ型と同等の効果を持たせることができる。

[モジュールを使ったスタックの記述例 (Modula-2)]

```
type stack;  
function pop(s: stack): integer;  
procedure push(a: integer; s: stack);  
function newstack: stack;
```

インタフェース

```
type stack = ↑rep;  
    rep = record ...  
function newstack: stack;  
    var s: stack;  
begin ...
```

実装

# (参考)抽象データ型

抽象データ型 (abstract data type) は、型とそれに付随する操作からなる。これは、言語が提供する型 (int 型など) と同様に新たな型をユーザが定義することができる機構である (ユーザ定義のデータ型)。

言語が提供する型 (int 型など) は、足し算、引き算、掛け算、割り算などの操作が付随しており、これらの操作を通じてのみ、その型の値にアクセスできるように言語が定義されていたら、異なるコンピュータで int 型等の内部表現が異なってもプログラムの振る舞いに影響が及ばない。

これと同様に抽象データ型の値を使うプログラムは抽象データ型の実装が変わっても影響を受けない。

# (参考)抽象データ型

Standard ML等が抽象データ型をサポートしている。Standard ML等の静的型付き言語では、抽象データ型が提供する操作を通じてのみ抽象データ型の値にアクセスできるように、型検査で制限することができる。(データ型の実装に依存するコードは型エラーとしてコンパイル時に検出される。)

[抽象データ型とモジュールとの比較]

抽象データ型では1つの型が定義されるが、moduleでは(相互に関連する)複数の型をひとまとまりにして扱える。

# C++におけるクラス

C++におけるクラスは、レコード(Cでは構造体と呼ばれる)を一般化したもの。

クラスを宣言した後はクラス名を型名として使用でき、その型の変数を宣言したり、オブジェクトを生成したりできる。

(例)

```
struct Stack {  
    int top, size;  
    char *elements;  
    Stack(int);  
    ~Stack();  
    char pop();  
    void push (char);  
};
```

```
class Stack {  
public:  
    int top, size;  
    char *elements;  
    Stack(int);  
    ~Stack();  
    char pop();  
    void push (char);  
};
```

# C++でのクラス宣言

```
struct X { <declarations> };
```

は、

```
class X { public : <declarations> }
```

と同じであり、

```
class X { <declarations> };
```

は、

```
struct X { private : <declarations> };
```

と同じである。

# クラス宣言の例

```
class Stack {  
    int top;  
    int size;  
    char *elements;  
public:  
    Stack (int n) {size=n;  
                    elements = new char[size]; top=0;}  
    ~Stack()      {delete elements; }  
    void push (char a) {top++; elements[top] = a;}  
    char pop()      {top--; return elements[top+1];}  
};
```

# オブジェクトの生成、破棄

C++ではオブジェクトはnewで生成し、deleteで破棄する。任意の型Tについて、

`new T`

によってT型のオブジェクトが生成され、それへのポインタが返される。

`delete p`

によって、pが指しているオブジェクトが破棄される。

(例) 前ページの例の、`elements = new char [size]`

によって、char型を要素とする長さsizeの配列が生成される。`elements[0]`, `elements[1]`, ..., `elements[size-1]`

によって配列の各要素が得られる。また、`delete elements`によって配列オブジェクトが破棄される。

# テンプレート(例で説明)

```
template <class T> class Stack {  
    int  top;  
    int  size;  
    T *  elements;  
public:  
    Stack (int n) {size=n; elements = new T[size]; top=0;}  
    ~Stack()      { delete elements; }  
    void push (T a) { top++; elements[top]=a; }  
    T  pop()      { top--; return elements[top+1]; }  
};
```

Stack型の変数を宣言したりオブジェクトを生成したりするとき、Stack<int> s(99); のように型を<>内に引数として与える。

# CとC++

C++は1983年、Bjarne Stroustrupによって設計、開発された。Cの拡張として設計されており、ほとんどのC言語のプログラムはC++のプログラムであり、意味も同じである。ただし、CとC++で意味が違うプログラムがある。

コメントは、Cでは `/* ... */` だが、C++では `// ...`  
(C99では `//` もコメントとして使えるが)

(例)

```
int f (int a, int b) {  
    return a /* * */ b  
    ;  
}
```

returnの右に書かれている式は、C89では `a/b`, C++およびC99では `a` となる。

# CとC++(続き)

C++では、構造体型を名前付きで宣言する構文で宣言した場合、その名前のみで構造体型を表せる。

(例1) `struct test {int a;}`  
`test x;`

のように書いてよい。Cでは、`struct test x;`と書く必要がある。(C++で`struct test x;`と書いてもよいが。)

(例2)

```
int x[99];
void f (void) {
    struct x {int a;};
    sizeof (x);
}
```

`sizeof(x)`は、Cでは配列`x`のサイズ、C++では構造体`x`のサイズ。Cでは、構造体`x`のサイズは`sizeof(struct x)`と書かなければならない。

# CとC++（続き）

Cでは、`sizeof('a')`は`sizeof(int)`と同じである。

C++では、`sizeof('a')`は`sizeof(char)`と同じである。

Cでは、列挙型のサイズは`sizeof(int)`である。

C++では、列挙型のサイズは、処理系依存である。

（列挙型の例）

```
enum color {RED, BLUE, YELLOW};
```

のように列挙型`color`を宣言すると、Cでは  
`sizeof(enum color)`は`sizeof(int)`と同じ、C++では処理系依存。

# オブジェクト指向

オブジェクト指向はシミュレーションを記述することを意図して考え出された。シミュレーションの中の要素がオブジェクトである。

(例) Simula (Simulation language, 1967)

- Ole-Johan Dahl, Kristen Nygaardが開発
- 最初のオブジェクト指向言語(オブジェクト指向という言葉はまだなかったが、オブジェクト指向における種々の概念が含まれていた)
- ALGOLの拡張として設計された
- 空港のシステムが例として記述された

# オブジェクト指向

[オブジェクトの例] 乗客、航空会社のカウンター、  
行列、チケット、...

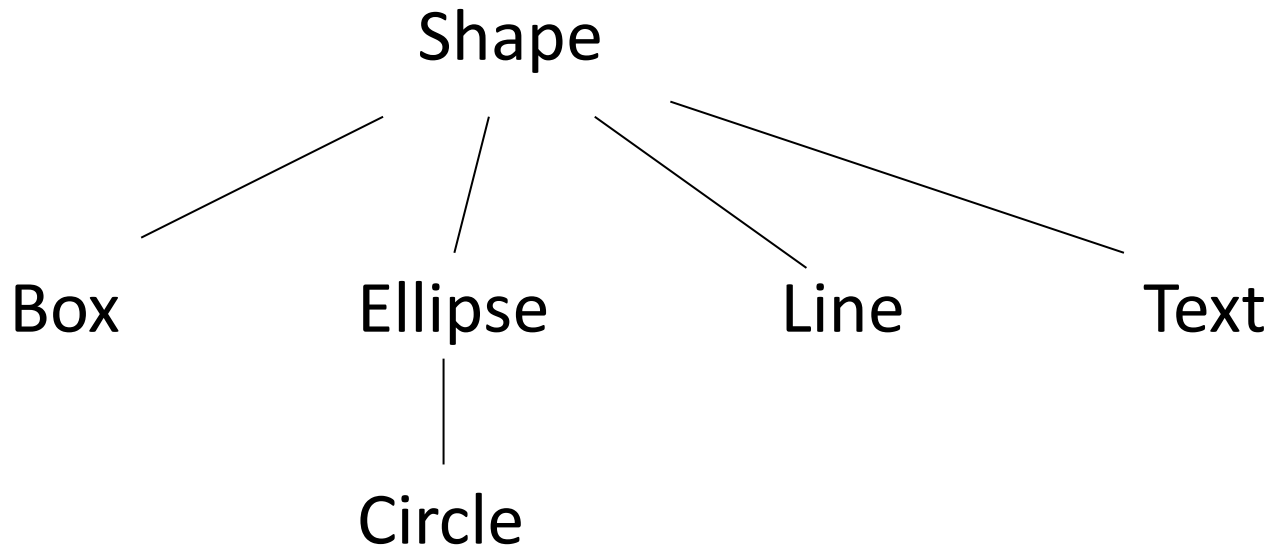
(外側から見た場合)

オブジェクト間でメッセージをやりとりすることにより  
計算が進んでいく。

(内側から見た場合)

メッセージを受け取ったら、それに対応する手続き  
を実行する。この手続きのことを、メソッド(method)  
あるいはメンバ関数(member function)という。

# クラスの階層構造



Shape

Box

Ellipse

Circle

Line

Text

# 継承(inheritance)

- 子クラスは親クラスのメソッド、変数を継承する(親クラスのメソッド、変数が子クラスのメソッド、変数になる)。
- 子クラスでは、追加でメソッドや変数を定義できる。同じ名前の場合には上書き(override)される。

(注意) overloadはoverrideとは全く別の概念。

Overloadは、引数の数や型が違うメソッドに同じ名前をつけること。

# C++の例

C++では、継承は以下のように記述する。

```
class Box : public Shape {  
    ...  
}
```

すべてのメンバーを  
visibilityを保って継承  
する。

```
class Box : private Shape {  
    ...  
}
```

継承されたメンバーは  
defaultでprivateメン  
バーになる。

C++では親クラスを基底クラス(base class)、子クラスを派生クラス(derived class)という。

# 仮想関数(virtual function)

メソッド宣言にvirtualというキーワードをつけると、  
(コンパイル時でなく)実行時にメソッドが選択される。

```
#include <stdio.h>
class Shape {
public:
    virtual void draw (void) {
        printf ("Shape¥n");
    };
};
```

```
class Box : public Shape{
    int num;
public:
    Box(int n) {num=n;}
    void draw (void) {
        printf ("Box %d¥n", num);
    }
};

int main (void) {
    Shape *s = new Box(1);
    s->draw();
    return 0;
}
```

Box 1  
が表示  
される

# 補足

メンバー関数は、(C++コンパイラ内で) this を引数に追加した関数へ変換すればよい。前ページの例の Box クラスのメンバー関数 draw は以下のように変換すればよい。

```
void draw(void) {printf ("Box %d¥n", num);}
```



```
void draw(Box *this) {printf ("Box %d¥n", this->num);}
```

これに合わせて、main 関数中の `s->draw()` を `draw(s)` に変換すればよい。`draw(s)` の呼び出しの中で `*s` のメンバー `num` にアクセスされる。`draw` は virtual なので、`s->draw()` は、オブジェクト `*s` のクラスに応じた `draw` が呼ばれるようにコンパイルされる。この例では Box クラスの `draw` が呼ばれ、Box 1 が表示される。

# C++の特徴

- Cとのbackward compatibilityをできる限り保ちつつオブジェクト指向をサポートするように設計された。
- オブジェクトをCの構造体の拡張とした。つまり、オブジェクトは関数のactivation recordやlocal block内にallocateされ得る。(もちろんヒープにもallocateできるが。)
- オブジェクト指向ではないプログラミング(命令型のプログラミング)もできる。(プログラミングのスタイルをプログラマに強要しない。)
- Multiple inheritanceをサポートする。(授業範囲外とする)

# オブジェクト指向言語のまとめ

オブジェクト指向言語(object-oriented language)は、オブジェクトを持ち、以下の4つの特徴を持つ。

- Dynamic lookup (メッセージを受け取ったときに実行されるメソッドは実行時に決まる)
- Abstraction (public関数がインターフェースとなり、データや実装は他の部分から見えない)
- Subtyping (基底クラスのオブジェクトが期待されている所ではそれをpublicに継承した派生クラスのオブジェクトが使える)
- Inheritance (継承。コード量が削減され、コードを修正しやすくなる。)

# 注意

Subtypingとinheritanceは異なる概念である。

(例)

Queue --- first-in, first-out

Stack --- last-in, first-out

Deque --- 両端から出し入れ可能なqueue

Dequeの派生クラスとしてQueueクラスとStackクラスを実装することができる(privateな継承を使って必要なメソッドのみpublicにすれば)。

しかし、Queue, StackはDequeのsubtypeではない。

# (参考) Data invariant

- 制御がオブジェクト内不在ときに常に成り立つ性質
- (例) bounded buffer (長さ制限付きqueue)
  - put(x), get()の2つのメソッドからなる。
  - 配列に要素を格納し、frontとrearで範囲を示す。  
配列の最後の要素の次は最初の要素とする。

- バッファはfrontとrearが等しいとき空
- Rearの次の要素がfrontのときバッファは一杯
- Frontとrearの間に、入力された順に要素が並んでいる。

オブジェクトは、data invariantを考慮しつつ設計する